BCSE307L · PROJECT A24 · TEAM 8 · BURS-BASED RETARGETABLE INSTRUCTION SELECTION

Code guide: which file does what

Every file in the repository, its job, its key functions with line numbers, and how the files connect.
github.com/ahmadazim542/burs-instruction-selector

1. The whole repository on one page

```
burs-instruction-selector/
├── Makefile                builds everything in 3 stages (burg → generate → link)
├── README.md               overview, commands, results, demo walkthrough
├── .gitignore              keeps build/ out of git
├── src/                    ← the team's code
│   ├── ir/
│   │   ├── ir.h            the IR node type: the contract every module shares
│   │   └── ir.c            builds, prints, parses and evaluates IR trees
│   ├── burs/
│   │   ├── target.h        what a "target machine" looks like to the engine
│   │   ├── engine.h        the engine's public functions
│   │   └── engine.c        the BURS engine: label, cost, cover, emit (+ helpers)
│   ├── targets/
│   │   ├── targetA/targetA.gr  Target A: register machine (rules, emitter, simulator)
│   │   └── targetB/targetB.gr  Target B: stack machine (rules, emitter, simulator)
│   └── main/
│       └── main.c          the burs program: command line, reports, suite, tests
├── tests/
│   ├── cases.txt           25 regression cases: id | options | expression
│   ├── expected/*.out      the exact output each case must produce (25 files)
│   ├── run_tests.sh        runs the cases + suite + exhaustive check (make test)
│   └── make_results.sh     regenerates results/ (make results)
├── results/                saved evidence: rule tables, sample runs, suite, tests, benchmark
├── docs/                   Review 1–3 documents, slides, notes, guidelines
├── third_party/burg/       BURG generator (not our code; built by the Makefile)
└── build/                  created by make, not in git: burg, gen/targetA.c, gen/targetB.c, burs
```

How one run flows through the files

What happens when you type `./build/burs "a + b * c"`:

1. Command line
main.c reads options and the expression
`main()`

2. Parse
ir.c turns text into an IR tree
`ir_parse()`

3. Label
engine.c asks the target's generated matcher to label every node
`burs_label()`

4. Cover
engine.c reads the best rules back from the root
`burs_print_cover()`

5. Emit
engine.c walks the cover; the target's .gr turns each rule into instructions
`burs_reduce()` → `reduceA()`

6. Check
the target's simulator runs the code; main.c compares with ir.c's value
`simulateA()` vs `ir_eval()`

Grey: machine-independent files (same for every target). Amber: target-specific code in `targetA.gr` / `targetB.gr`. Steps 3–6 repeat for Target B on the same tree.

How the program is built (`make`)

Stage	Command (from the Makefile)	Result
1	`cc … -o build/burg third_party/burg/*.c`	The BURG generator program.
2	`build/burg -p burmA -o build/gen/targetA.c targetA.gr` `build/burg -p burmB -o build/gen/targetB.c targetB.gr`	Two C files: each is a generated matcher (state tables + `label`, `rule`, `nts`, `kids`) followed by the grammar's own C code, copied unchanged. The prefixes stop the two from clashing.

3	`cc -std=c99 … -o build/burs ir.c engine.c main.c gen/targetA.c gen/targetB.c`	The single `burs` program containing both targets.

2. Source files, one by one

`src/ir/ir.h`

83 lines · IR module **Vineet**

Defines the intermediate representation: the tree node every other file uses. It is the single contract between the parser, the engine and both targets.

Item	Line	What it is
`enum { CONST=1 … LOAD=6 }`	14	The six IR operators. The numbers must match the `%term` line in both grammars.
`struct node`	28	One tree node: `op`, `state_label` (written by the matcher), `id` (n1, n2…), `ival` (constant), `name` (variable), `left`, `right`.
`OP_LABEL`, `STATE_LABEL`, `LEFT_CHILD`, `RIGHT_CHILD`	38–41	Accessor macros the burg-generated matchers use to read and label the tree.
function prototypes	43–81	Declarations of everything in `ir.c` (builders, printers, parser, evaluator).

Used by: every other source file.

`src/ir/ir.c`

401 lines · IR module / front end **Vineet**

Everything about IR trees that does not depend on any machine: building nodes, printing them, parsing text into a tree, and computing the "correct answer" used to check generated code.

Function	Line	What it does
`ir_opname`, `ir_arity`	18, 23	Name ("ADD") and number of children of an operator.
`mk_const`, `mk_var`, `mk_add`, `mk_sub`, `mk_mul`, `mk_load`	46–63	Tree builders: allocate one node each.
`freetree`	65	Frees a whole tree.
`ir_number`	84	Numbers nodes n1…nN in postorder (the ids shown in covers and cost tables).
`printtree`, `ir_draw`	102, 143	One-line prefix form `ADD(VAR a, MUL(…))` and the indented tree drawing.
`parse_primary`, `parse_term`, `parse_expr`	212, 277, 294	Recursive-descent parser: numbers, names, `( )`, `[e]` / `load(e)`; `*` before `+ −`; left-associative. Produces the error messages ("column 7: expected ')'…").
`ir_parse`	310	Entry point: text → tree, or `NULL` plus an error message.
`ir_set_var`, `ir_var_value`, `ir_mem_read`	354, 370, 385	Test values: a = 2, b = 3, …; `-D` overrides; memory model `mem[x] = 1000 + x`.
`ir_eval`	390	Evaluates the tree directly. This is the reference value every generated program must match.

Used by: `main.c` (parse, print, evaluate), both simulators (variable values, memory).

src/burs/target.h

82 lines · engine ↔ target interface **Utsav**

Describes what a target machine must provide. The engine only ever talks to targets through the `Target` struct defined here, which is what makes the system retargetable.

Item	Line	What it is
`BURS_MAX_KIDS / _INSTR / _INSTR_LEN`	18–20	Limits: up to 512 instructions of 48 characters per program.
`PANIC`	23	Error hook required by the generated matcher.
`Operand`	26	What a reduced subtree turns into: `"r3"`, `"#4"`, `"[a]"` or top of stack.
`EmitCtx`	31	The output buffer: the instruction lines, the next free register, an overflow flag.
`RuleInfo`	39	One rule's printable pattern, cost and instruction template.
`struct Target`	45	Everything about one machine: name, start nonterminal, rule table, the four generated matcher functions, `reduce()` and `simulate()`.
`targetA`, `targetB`	66–67	The two targets, defined at the end of each `.gr` file.
helper prototypes	70–80	`burs_emit`, `burs_operand`, `burs_new_reg`, `burs_split` (implemented in `engine.c`).

Used by: `engine.c`, `main.c`, both `.gr` files.

src/burs/engine.h

30 lines · BURS engine API **Utsav**

Declares the six engine functions `main.c` calls (lines 13–28). Each takes a `const Target *`, so the same code works for any machine.

src/burs/engine.c

205 lines · BURS engine **Utsav** **helpers: Ahmad**

The machine-independent BURS driver. It never names an instruction: it labels the tree through the target's generated matcher, reads costs and the cover back, and drives code emission.

Function	Line	What it does
`burs_label`	15	Bottom-up labeling: calls the generated `label()`, which walks the tree in postorder and stores a state in each node (one table lookup per node). Returns whether the root can be covered.
`burs_cost`	21	Cost of the best way to compute nonterminal *nt* at a node: the rule's cost plus its children's costs.
`print_cover` / `burs_print_cover`	38, 64	Backtracks from (root, start nonterminal) and prints the selected rules top-down ("Selected cover").
`cost_table` / `burs_print_cost_table`	69, 96	The −v table: for every node and nonterminal, the best rule and its cost.
`burs_reduce`	103	Code emission: for the selected rule, reduces the children first (left to right), then calls the target's `reduce()`. Output comes out in postorder.
`burs_print_rules`	121	Prints a target's rule table (`--rules`).
`burs_emit`, `burs_operand`, `burs_new_reg`	138, 151, 162	Helpers for emitters: append one instruction (stops at 512), build an operand string, allocate the next virtual register.
`burs_split`	171	Helper for simulators: splits `"ADDI r2,r1,#4"` into opcode and operands.

Calls: the target's generated matcher and its `reduce()`. **Called by:** `main.c`.

src/targets/targetA/targetA.gr

184 lines · Target A, register machine

rules: Mayank

emitter + simulator: Ahmad

Everything about Target A, a 3-address load/store machine (LOAD, LOADI, ADD, ADDI, SUB, SUBI, MUL; unlimited virtual registers). The rules section is read by burg; the C after the second %% is copied into the generated file.

Part	Line	What it does
preamble %{ … %}	1–21	Comment describing the ISA; includes target.h.
%start reg, %term …	23–24	Start nonterminal and the six operators (same numbers as ir.h).
9 rules	27–35	nonterminal: PATTERN = rule-number (cost); e.g. reg: ADD(reg,imm) = 6 (1); is ADDI.
ntA[], rulesA[]	38, 44	Nonterminal names; printable pattern, cost and template for each rule.
reduceA	58	The emitter: given one selected rule and its children's operands, emits that instruction into a new register and returns it. imm: CONST emits nothing and returns #k.
simfail, regnum	95, 101	Simulator helpers: format an error; parse rN.
simulateA	114	Runs the generated text on a register file, rejects undefined registers, returns the result register's value.
const Target targetA	176	Packs all of the above (plus the generated burmA_* functions) into one object.

src/targets/targetB/targetB.gr

192 lines · Target B, stack machine

rules: Mayank

emitter + simulator: Ahmad

Everything about Target B, a 0-address stack machine with memory (ADDM [x]) and immediate (ADDI #k) operand forms. Same structure as Target A.

Part	Line	What it does
%start stk, %term …	23–24	Start nonterminal and the same six operators.
14 rules	27–40	Nonterminals stk (on the stack), mem (a variable operand), imm (a constant). Chain rules stk: mem / stk: imm are PUSH / PUSHI.
ntB[], rulesB[]	43, 50	Nonterminal names and the printable rule table.
reduceB	69	The emitter: mem and imm return operands; PUSH rules push them; ADD/SUB/MUL work on the stack; …M and …I forms take the right operand directly.
operand_value, arith	112, 129	Simulator helpers: decode [x] / #k; do +, −, ×.
simulateB	134	Runs the code on a value stack; checks underflow, operand kinds, and that exactly one value is left.
const Target targetB	184	Packs Target B into one object.

src/main/main.c

545 lines · driver and test modes **Ahmad**

The `burs` program itself. It reads the command line, runs the expression through every selected target, prints the report, and provides the suite, exhaustive, benchmark and interactive modes.

Function / item	Line	What it does
`EXIT_USAGE / _PARSE / _CHECK`	15–17	Exit codes 1, 2, 3 (0 = success).
`all_targets[]`, `suite[]`	19, 23	The targets to run (add a new target here) and the 10 review expressions.
`usage`	52	The –h help text.
`collect_vars`, `print_vars`	72, 89	The "Test values" line.
`run_target`	110	The pipeline for one target: label → cost table (–v) → cover → emit → print code → simulate → compare.
`process`	180	One expression: parse, print the IR, run every target, print the retargeting summary.
`run_suite`	227	`--suite`: the 10 expressions as a PASS/FAIL table.
`naive_cost`, `run_exhaustive`	275, 289	`--exhaustive`: all 3,333 trees of depth ≤ 2 on both targets, plus the naive-selector comparison.
`random_tree`, `count_nodes`, `run_bench`	361, 379, 384	`--bench`: times labeling on random trees from 405 to 239,710 nodes.
`interactive`	425	`-i`: one expression per line until `quit`.
`set_var`	450	Handles `-D name=value`.
`main`	465	Parses options (`-t`, `-v`, `-D`, `-i`, `--rules`, `--suite`, `--exhaustive`, `--bench`) and picks the mode.

3. Build, tests and results

File	Owner	What it does
`Makefile`	Vineet	Targets: `make` (3-stage build into `build/`), `make test`, `make demo`, `make results`, `make clean`.
`tests/cases.txt`	Ahmad	25 regression cases, one per line: `id | options | expression`. Grouped as positive (`pos_`), boundary (`bnd_`), rule choice (`rule_`) and negative (`neg_`).
`tests/expected/<id>.out`	Ahmad	The exact output each case must produce, including `exit=N` on the last line. 25 files.
`tests/run_tests.sh`	Ahmad	Runs every case and compares its output byte for byte with the expected file, then runs `--suite` and `--exhaustive`. Prints `27 passed, 0 failed`. `--update` rewrites the expected files after a deliberate change.
`tests/make_results.sh`	Ahmad	Regenerates everything in `results/`.
`results/rule_tables.txt`	generated	Output of `--rules`: both rule tables.
`results/sample_runs.txt`	generated	Full –v output for the six review expressions.
`results/suite_summary.txt`	generated	Output of `--suite` (10/10 PASS).
`results/exhaustive_check.txt`	generated	Output of `--exhaustive` (6,666 runs, 0 failures; naive 7.26, A 6.88, B 5.55).
`results/benchmark.txt`	generated	Output of `--bench` (time per node vs tree size).

`results/test_report.txt`	generated	Output of `run_tests.sh`: 27 PASS lines.
`README.md`	Ahmad	Project overview, commands, demo walkthrough, rule tables, results.
`.gitignore`	–	Keeps `build/` out of the repository.

4. Documents (`docs/`)

File	What it is
`Review1_Presentation.pptx`, `Review1_Document.docx`	Review 1: problem, design, plan.
`Review2_Document.pdf`	Review 2: implementation progress (Target A).
`Review3_Guidelines.pdf`	The course's Review 3 guidelines.
`Review3_Document.docx` / `.pdf`	Review 3 final report.
`Review3_Presentation.pptx` / `.pdf`	Review 3 slides with speaker notes.
`Review3_Presenter_Notes_Ahmad_Azim.pdf`	Ahmad's presenter notes and viva preparation.
`Code_Guide.pdf`	This document.

5. Third-party: `third_party/burg/`

BURG is the code-generator generator by Fraser, Hanson and Proebsting, in Chen Zhexuan's modernised version. It is **not team code**: it is included unmodified so `make` can build it, and it runs only at build time to turn each `.gr` file into a C matcher. Its licence is in `COPYRIGHT` and `LICENSE.TXT`.

Reads the grammar

`main.c` (command line: `-p`, `-o`), `lex.c` (tokenizer), `gram.y` → `gram.tab.c/.h` (grammar parser generated by Bison), `fe.c`, `fe.h` (declarations, rules, patterns).

Computes the BURS states

`burs.c`, `item.c`, `closure.c` (chain rules), `delta.c` (cost normalisation), `map.c` (state map), `queue.c` (work list), `trim.c`, `table.c` (transition tables).

Writes the C matcher

`be.c` (writes `label`, `rule`, `kids`, `nts` and the tables), `plank.c` (compresses the tables).

Utilities and other files

`b.h`, `list.c`, `string.c`, `symtab.c`, `rule.c`, `pattern.c`, `operator.c`, `nonterminal.c`, `zalloc.c`; `sample.gr`, `doc.tex`, READMEs, `CMakeLists.txt` (unused by our Makefile).

6. "Where do I look if…?"

You want to…	Look at
change an instruction's cost	the rule line in `targetA.gr` (27–35) or `targetB.gr` (27–40) *and* the matching `rulesA[]` / `rulesB[]` entry
change how an instruction is printed	`reduceA` (targetA.gr:58) or `reduceB` (targetB.gr:69)
change an error message for bad input	the parser in `ir.c` (212–340)
change test variable values or the memory model	`ir_var_value` / `ir_mem_read` in `ir.c` (370, 385)
change what the report prints	`run_target` / `process` in `main.c` (110, 180); cover and cost table in `engine.c`
add a command-line option	`main` in `main.c` (465) and `usage` (52)

add a test case	a line in `tests/cases.txt`, then `sh tests/run_tests.sh --update` and review the new `.out` file
add a new IR operator	`ir.h` enum + `ir.c` (parser, printer, `ir_eval`) + the `%term` line and rules in both grammars
add a third target machine	new `src/targets/targetC/targetC.gr` (same `%term`; rules, `reduce`, `simulate`, `const Target targetC`), a `burg -p burmC` rule in the `Makefile`, `extern` in `target.h`, and `&targetC` in `all_targets[]` (main.c:19)

7. Function calls for `./build/burs "a + b * c"`

```
main()                                      main.c:465   options → one expression
└─ process()                                main.c:180
   ├─ ir_parse()                             ir.c:310     text → tree, nodes numbered n1..n5
   ├─ printtree(), ir_draw(), print_vars()   ir.c / main.c
   ├─ ir_eval()                              ir.c:390     reference value = 14
   └─ for each target in all_targets[]:
      └─ run_target()                        main.c:110
         ├─ burs_label()                     engine.c:15  → burmA_label() / burmB_label() (generated)
         ├─ burs_cost()                      engine.c:21  total cover cost
         ├─ burs_print_cover()               engine.c:64  "Selected cover"
         ├─ burs_reduce()                    engine.c:103 children first, then…
         │  └─ reduceA() / reduceB()         targetX.gr   …emit one instruction per rule
         └─ simulateA() / simulateB()        targetX.gr   run the code; must equal 14
   └─ retargeting summary                    main.c       A: 5 instr, cost 6 · B: 4 instr, cost 5
```